B-PROG2, 2026, RT Meno: [redacted]

Pozorne čítajte zadanie, čitateľne napíšte kód, alebo požadované časti kódu/opravy, výstupy kódu podľa zadania. Max. 5 bodov / úlohu.

1. Vstupom funkcie je konštantný reťazec `str`. Funkcia alokuje nový reťazec a naplní ho iba písmenami z pôvodného reťazca - využite funkciu `isalpha`.
*Príklad: only_alphas(" a2-3 *b! ") vráti "ab".*

```
char* only_alphas( const char * str ) {
    int length = strlen( str);
    int j = 0;
    char* buffer = malloc( length);
    for( int i = 0; i < length; i++){
        if ( isalpha( str[i])){
            buffer[ j ] = str[i];
            j++;
        }
    return buffer;
  }
}
```

2. Vstupom funkcie je pole `n` reťazcov. Funkcia nájde najdlhší reťazec v poli a vráti ho - využite funkciu `strlen`. Ak je ich viac, vráti prvý z nich.
*Príklad: top(4, {"a2-3", "*b!", "longstring", "0123456789"}) vráti "longstring".*

```
char* top( int n, char * pole [] ) {
    int length = strlen (pole[0]);
    char* vysl;
    int index;
  for ( int i = 0; i < n; i++){
      if ( length < strlen( pole[i]) ) {
          length = strlen(pole[i]);
     index = i; ~~vysl = &pole[i];~~ ~~strcpy(vysl~~
      }
      strcpy( vysl, pole[index]);
   return vysl;
}
```

[✓ Súhlasím] / *Nesúhlasím so spracovaním odovzdanej skúšky na výskumné účely (výskum využitia AI na hodnotenie písomiek).*

B-PROG2, 2026, RT

Meno: [redacted]

3. Vstupom funkcie je matica. Funkcia zistí počet nulových riadkov matice.
Príklad: zero_rows(3, 4,[[0,0,0,0], [0,1,0,0], [0,0,0,0]]) vráti 2.

```
int zero_rows( int rows, int cols,
                         double M[rows][cols] ) {

    int condition = 1;
    int count = 0;
   for ( int i = 0; i < rows; i++) {
        condition = 1;
      for( int j = 0; j < cols; j++){
           if (M[i][j] != 0) {
              condition = 0;
               break;
           }
       }
      if ( condition ) {
           count++;
      }
    }
   return count;
  ~~}~~
}
```

4. Skompletizujte funkciu `swap`. Napíšte funkciu, ktorá nájde najmenší prvok v poli (prvý v poradí ak ich je viac) a pomocou `swap` ho prehodí s prvým prvkom v poli.
Príklad: min_first(5, [5, -7, -12, 8, -2]) zmení data na [-12, -7, 5, 8, -2].

```
void swap( int *p1, int *p2 ) {
        int p = *p1;
         *p1 = *p2;
          *p2 = p;
}

void min_first( int size, int data[size] ) {
    int min = data[0];
    int i = 0;
    int index = 0;
   for ( i; i < size; i++) {
      if ( min > data[i]) {
          min = data[i];
          index = i;
        }
     }
    swap (&data [index]; &data[0]);
   ~~}~~
}
```

Meno: [redacted]

5. Vstupom funkcie je meno textového súboru na disku. Funkcia má načítať jeho obsah do dynamicky alokovaného reťazca. Doplňte chýbajúce časti kódu podľa komentárov.

```
char* read_text_file( const char * filename ) {  ~~BUFFER!~~

  FILE *f; char *str = NULL;

  // otvor subor na citanie

  f = fopen(filename, "r");

  // ak sa nepodarilo otvorit subor, vrat NULL pointer

  if (f == NULL)

     return NULL;

  // zistime velkost suboru
  fseek( f, 0, SEEK_END );
  size_t size = ftell(f);
  rewind (f);

  // alokujeme dostatocnu oblast v pamati

  str = (char*) malloc(1+size);

  if (str != NULL) {
     // nacitame a ukoncime retazec str
     size_t bytes_read = fread(~~buffer~~, 1, size, f);
     ~~bytes_read~~ str[size] = '\0';
  }
  //uzavrieme subor
  fclose(f);
  return str;
}
```

6. Úlohou funkcie je načítať riadok z otvoreného súboru, využívajúc pritom funkciu realloc na alokáciu dostatočnej pamäte (predpokladáme jej dostatok).
Vo funkcii sú viaceré chyby. Opravte ich zakomentovaním chybného riadku, a ak je to potrebné, napísaním opravy pod zakomentovaný riadok, alebo doplnením chýbajúceho riadku kódu.

```
char* readline( FILE * f ) {
   /int size = 1024, c, ix = 0;
    ~~[illegible]~~

    char buffer[size];

  //fopen ( f );                                   // pre istotu :)
  FILE * f = fopen ( f , "r"); if (f == NULL)
    while ( (c = fgetc(f)) != EOF ) {          return NULL;

       if (ix == size - 1) {

          size += 1024;

          realloc( buffer, size );

       }
    // if ( c == '\n' ) break;
    // nie je definované so zadaním, že sa má skončiť
       buffer[ix] = c;      // načítanie pri '\n'

        ix++;
    }

    return buffer;
}
```

[✓ Súhlasím] / *Nesúhlasím so spracovaním odovzdanej skúšky na výskumné účely (výskum využitia AI na hodnotenie písomiek).*

Meno: [redacted]

7. Používame štruktúru z hry Monopoly:

```
typedef struct _property {
    char name[MAX_NAME + 1];     // nazov
    int price;                   // cena nehnutelnosti
    COLOR color;                 // farba
} PROPERTY;
```

Napíšte funkciu, ktorá nájde v poli nehnuteľnosť so zadaným názvom (HINT: využite `strcmp`). Ak sa tam taká nenachádza, vráti NULL.

```
PROPERTY* find_property( PROPERTY props[NPROP],
              const char* name ) {
        for( int i=0; i<NPROP; i++){
              if ( strcmp( props[i].name, *name){
                     return props[i];
                   }
              }
          return NULL;
}
```

8. Napíšte funkciu, ktorá vypočíta celkovú hodnotu majetku hráča hry Monopoly. Jedná sa o súčet jeho hotovosti a ceny všetkých vlastnených nehnuteľností. PROPERTY štruktúra je podľa úlohy 7. Relevantné údaje o hráčovi sú uložené v nasledovnej štruktúre:

```
typedef struct _player {
    int cash;                   // hotovost hraca
    PROPERTY *owned[NUM_PROPERTIES]; // zoznam nehnutelnosti
    int num_props;                   // pocet  nehnutelnosti
} PLAYER;
```

```
int player_value( PLAYER *player ) {
      int cash = 0;
      cash += player->cash;
      for ( int i = 0; i < num_props; i++){
           cash += player->owned[i]*→ price;
                                       //šípka !
           }
       return cash;
}
```